Objectifs RTO / RPO

> ***Statut : cible.*** *Ce document décrit ce que l'équipe vise et a décidé, pas ce qui est déjà codé. L'état réel du code est suivi dans `constat/etat-implementation.md`.*

> *Ticket : WOR-70 · Responsable : Cédric · Version 0.3 du 21/10/2026*
>
> *Objectifs fixés à partir du coût d'un arrêt (`services-critiques.md`) et des seuils de DORA (`nis2-dora.md`), avec **la façon de les mesurer** pendant les essais (WOR-79).*

1. Définitions retenues

- **RTO** (durée maximale d'interruption) : temps entre le **début de l'impact** (première transaction en échec) et la **première transaction réussie et vérifiée** après la reprise. On compte depuis l'impact, pas depuis la détection : c'est ce que subissent les partenaires, et ce que DORA mesure.
- **RPO** (perte de données maximale admissible) : quantité de **transactions confirmées au client** qui manquent ou sont dupliquées après la reprise. On l'exprime en nombre de transactions.
- **Reprise vérifiée** : API accessible, transaction test réussie, données cohérentes (WOR-77). Un conteneur redémarré ne compte pas.

2. Deux niveaux d'objectifs

Niveau	À quoi il sert	D'où il vient
Objectif métier	Ce que l'entreprise fictive exige ; c'est ce qu'on présente à la direction	Coût d'un arrêt : au-delà de 5 min, l'incident sort de G1 ; au-delà de 30 min, l'incident passe en G3 : le budget SLA de 43 min par mois est presque consommé (pénalité d'environ 650 k€ au-delà) et la notification DORA devient probable
Cible du démonstrateur	Ce qu'on vise et mesure sur le PC Hôte	Objectif métier, avec de la marge, et la contrainte du Clip Chrono (vidéo continue de 60 s exigée par le sujet) : 60 s en continu entre l'attaque et le service rétabli (WOR-83)

3. Objectifs par service

Service	Criticité	RTO métier	RPO métier	Cible démonstrateur	Mécanisme prévu
Traitement des transactions (API + BDD)	Vitale	**≤ 5 min**	**0** transaction confirmée perdue ou dupliquée	RTO ≤ 60 s (attaque A) · ≤ 3 min (attaque B)	Remédiation automatique (WOR-38), playbooks Ansible (WOR-44)
Authentification	Vitale	≤ 5 min	Sans objet : une session perdue oblige seulement à se reconnecter	Retour en même temps que l'API	Redémarrage ; après une rotation de secret, reconnexion des utilisateurs acceptée
WAF / point d'entrée	Vitale	≤ 5 min	Sans objet ; la configuration est versionnée	Pas de coupure pour un blocage d'IP	Liste d'IP bloquées rechargée à chaud (20 s maximum) ; les autres règles demandent un redémarrage
Journalisation et SIEM	Importante	≤ 1 h	**0** journal perdu	Aucune perte pendant les essais	Export des journaux (`scripts/export-logs.sh`) **avant** toute restauration

Service	Criticité	RTO métier	RPO métier	Cible démonstrateur	Mécanisme prévu
Consultation de l'historique	Importante	≤ 30 min	0	Retour en même temps que l'API	Même service que l'API
Supervision, détection ML, remédiation	Support critique	≤ 15 min	Sans objet ; le modèle est versionné	Hors périmètre de l'attaque	Relance par Docker Compose
Assistant RAG	Non critique	Meilleur effort	Sans objet ; l'index peut être reconstruit	—	Réindexation

4. Répartition du RTO sur la chaîne de reprise

Le RTO se décompose en étapes. Chaque pilier tient une partie du budget. Les durées sont des **hypothèses à confirmer** avec Reza (collecte), Léa (détection) et Michael (remédiation).

Étape	Marqueur de temps	Responsable	Budget attaque A	Budget attaque B
Début de l'impact → anomalie visible dans les métriques	T1 → T2	Infrastructure (collecte Prometheus toutes les 5 s)	≤ 5 s	≤ 5 s
Anomalie visible → alerte ML	T2 → T3	IA / Data (fenêtre de 60 s évaluée toutes les 10 s, alerte après trois fenêtres anormales consécutives)	≈ 30 s	≈ 30 s
Alerte → décision de remédiation	T3 → T4	Développement	≤ 5 s	≤ 5 s
Décision → action terminée	T4 → T5	Infrastructure / Développement	≤ 15 s (blocage d'IP au WAF ; redémarrage de l'API si besoin, jusqu'à 90 s)	≤ 2 min (blocage, recréation du conteneur)
Action terminée → transaction test réussie et données vérifiées	T5 → T6	Développement	≤ 10 s (jusqu'à 6 essais espacés de 3 s)	≤ 20 s
Total (RTO mesuré = T6 – T1)			**≤ 60 s**	**≤ 3 min**

T0 désigne le lancement de l'attaque sur le PC Attaquant. L'écart T0 → T1 dépend de l'attaque et ne compte pas dans le RTO.

> ***Point de vigilance pour le Clip Chrono :*** *60 s, c'est T0 → T6* ***et*** *l'affichage du résultat. La collecte passe déjà toutes les 5 s (`infra/prometheus/prometheus.yml`). La détection reste le poste le plus long : environ 30 s si trois évaluations consécutives suffisent, jusqu'à 80 s si la fenêtre de 60 s doit se remplir d'abord (`docs/runbooks/reprise.md`). Le passage de 2 à 3 fenêtres (21/10, pour réduire les fausses alertes) ajoute 10 s : la somme des budgets de l'attaque A atteint 65 s et dépasse les 60 s.* ***À mesurer en priorité avec Léa.***

5. La restauration après une injection (attaque B)

Ce que fait le code : le moteur recrée le conteneur de l'API, qui garde ses données en mémoire. Le service revient vite, mais **toutes les transactions sont perdues** (RPO non tenu). C'est le choix retenu

pour l'instant ; les options ci-dessous restent celles à étudier pour tenir le RPO = 0.

Le scénario d'attaque (WOR-29) prévoyait un **retour à une copie instantanée (snapshot) propre de la base**. C'est incompatible avec un RPO de 0 : toutes les transactions légitimes validées entre le snapshot et la restauration seraient perdues.

Option	RPO obtenu	Effort	Avis
1. Snapshot périodique seul (par exemple toutes les 5 min)	Jusqu'à 5 min de transactions perdues	Faible	Simple, mais contredit l'objectif métier et provoque un incident majeur DORA (perte de données)
2. **Snapshot + rejeu des transactions légitimes** depuis un journal en ajout seul, en écartant celles qui viennent de la source bloquée	**0** si le rejeu est correct	Moyen	**Recommandé** : démontre l'objectif métier, et le tri entre légitime et malveillant est un bon argument pour le jury
3. Restauration à un instant précis (avec PostgreSQL : archivage continu du journal des écritures), juste avant la première requête malveillante	Proche de 0, mais les transactions légitimes postérieures sont perdues	Moyen à élevé	Solution industrielle de référence, à citer dans les recommandations
4. Correction ciblée : annuler les seules lignes altérées, sans restauration globale	0	Élevé et risqué	Non retenu pour un prototype de 4 jours

Proposition : option 2 si l'équipe a le temps de brancher l'API sur PostgreSQL, option 3 citée comme cible industrielle. Dans tous les cas, **les journaux sont copiés avant la restauration** (`scripts/export-logs.sh`) : ce sont des preuves pour l'investigation et la notification.

6. Protocole de mesure (pour WOR-79)

Temps

Chaque marqueur est horodaté automatiquement, sur une **même horloge** (le PC Hôte ; le PC Attaquant est synchronisé dessus ou son décalage est noté avant chaque essai).

Marqueur	Source
T0 lancement de l'attaque	Journal du script d'attaque (WOR-58)
T1 première transaction en échec	Générateur de trafic normal : première réponse en erreur ou en dépassement de délai
T2 anomalie visible	Prometheus : premier point hors norme
T3 alerte ML	Journal du service d'inférence (WOR-53)
T4, T5 décision, action terminée	Événements publiés par le moteur de remédiation (WOR-38)
T6 reprise vérifiée	Moteur de remédiation : transaction test et contrôle de cohérence réussis

Données

1. Le générateur de trafic enregistre chaque transaction **confirmée** (identifiant, montant).
2. Après la reprise, on compare cette liste au contenu de la base.
3. **Transactions manquantes + transactions en double = RPO mesuré.** L'objectif est 0.

Grille de résultats

Trois essais minimum par attaque pendant les répétitions (WOR-78) ; détail des essais dans le journal du moteur.

Attaque	Détection T1 → T3	Reprise T3 → T6	RTO mesuré (pire des 3)	Transactions perdues ou dupliquées	Objectif tenu ?
A — déni de service	≈ 30 s (réglage : 3 fenêtres)	~7 s	**≈ 40 s**	0 (reprise par blocage, sans redémarrage)	Oui : ≤ 60 s (démo) et ≤ 5 min (métier)
B — injection	—	—	—	—	Non chronométrée : l'attaque vise une route inexistante

Résultats des répétitions du 22/10. La détection n'a pas été chronométrée séparément : la valeur est celle qu'impose le réglage du modèle.

On retient le **pire des trois essais**, pas la moyenne : un RTO se garantit, il ne s'espère pas.

7. Ce que le code permet de tenir au 22/10

- **RPO = 0 : pas tenable en l'état.** Dans le socle, l'API garde ses données en mémoire : un redémarrage (attaque A) ou une restauration (attaque B) efface toutes les transactions. Aucune sauvegarde.
- **Attaque A : reprise totale ≈ 40 s** aux répétitions du 22/10 (≈ 30 s de détection, ~7 s de reprise), sans transaction perdue ni intervention humaine (§6).
- **Vérification de la reprise (T6) : en place.** Le moteur passe une transaction de test complète à travers le WAF.
- **Durée affichée par le moteur :** elle part de la réception de l'alerte (T3 → T6). Le RTO se calcule depuis T1, avec le protocole du §6.

Détail et sources : `constat/etat-implementation.md`.

8. Ce que ces objectifs ne couvrent pas

- **Perte du PC Hôte** (panne, rançongiciel R7) : pas de deuxième site, donc pas de RTO tenable. La reprise passe par une réinstallation depuis le dépôt Git (WOR-49), sans objectif chiffré. C'est un risque résiduel assumé, à porter dans le PCA/PRA (WOR-71).
- **Sauvegardes sur le même hôte** : le RPO de 0 ne tient que si la base n'est pas détruite avec son hôte.
- **Faux positif du modèle ML** : une remédiation déclenchée à tort provoque elle-même une coupure, que le RTO doit absorber. C'est arrivé le 21/10 : deux fausses alertes d'injection ont déclenché deux restaurations (`ml/training/README.md`). À suivre pendant le chaos engineering (WOR-76).